AdaptLearn

AI-Based Personalized Learning Content Generator

Python • Streamlit • Ollama (Llama 3.2 3B)

Your Name | College / Course | Date

Beginner

Intermediate

Advanced

One size does not fit every learner

Traditional platforms give everyone the same content. Example: three students learning Python loops.

Beginner

Simple definitions and everyday examples.

Intermediate

Code examples and common mistakes.

Advanced

Deeper concepts, edge cases, challenges.

Main objectives

1. Personalize content by proficiency
2. Short revision notes
3. Auto-generate practice questions
4. Identify concepts answered wrongly
5. Adjust difficulty from quiz scores
6. Show progress on a dashboard
7. Run AI locally, no paid API key

Who uses it, and what the dashboard offers

College students

Learn programming, maths, data structures, AI/ML at different levels.

Teachers

Generate explanations, notes and sample quizzes for demos.

Self-learners

Practise, check understanding, find weak areas.

Potential users based on features, not evidence of deployment or testing.

Tab 1: Learn

Explain a topic, make notes, take a 5-question quiz, review answers.

Tab 2: Compare levels

Beginner, Intermediate and Advanced explanations side by side.

Tab 3: Progress

Scores, trend chart, attempt table, knowledge gaps.

Tab 4: Prompt & logic

Prompt template, level rules, grading and adjustment.

Sidebar: Topic of interest • Self-assessed level • Current level • Reset progress

How the AI personalization works

Level 1 | Beginner

Very simple language, a familiar analogy, short steps, key terms defined.

e.g. a loop as repeating a daily task.

Level 2 | Intermediate

Clear definitions, one worked example, two common mistakes.

e.g. Python code and off-by-one errors.

Level 3 | Advanced

Formal details, edge cases, trade-offs, a challenge problem.

e.g. loop complexity and performance.

The STYLE dictionary is combined with the topic, recent quiz scores and recorded weak concepts.

How difficulty adapts to quiz scores

80–100%

Move up one level (max Advanced)

41–79%

Stay at current level

0–40%

Move down one level (min Beginner)

Example: a student learns "Stack in Python"

1. Initial level

Beginner; requests an explanation.

2. Take the quiz

Scores 4/5 = 80%.

3. Adjust

Moves up to Intermediate.

4. Continue

New material uses the new level and recorded gaps.

Illustrative example of the code's rules, not an actual test result.

Behind the scenes: architecture and tech stack

Student input
Topic + level

▶

Prompt builder
Level rules, scores, gaps

▶

Local model (Ollama)
Generates content

▶

Python grading
Score, gaps, level, progress

Outputs: streamed learning content, and quiz JSON that is parsed and validated.

Python	App logic
Streamlit	Web dashboard and controls
Ollama + Llama 3.2 3B	Local model (llama3.2:3b, localhost:11434)
Requests, JSON, JSON Schema	API calls, quiz structure
re, Session State, HTML/CSS	Clean fences, hold session data, styling

Why local (Ollama)?

No paid API key; runs on your computer; can work offline once the model is installed. Cloud AI may offer larger models but needs internet, keys and data-handling care.

No database, login system or external AI service is used currently.

How quiz generation works

generate_quiz(): 5 MCQs, each with

q – question text
options – four answers
answer – correct index (0–3)
why – explanation
concept – concept tested

1. Request quiz from Ollama with a JSON schema
2. Strip Markdown code fences
3. Extract list; check it has 5 questions
4. Validate text, 4 options, answer index, explanation
5. On failure, retry once for a corrected response
6. Display quiz; grade in Python

Schema output is not guaranteed valid. Known gap: a missing concept is not rejected, so its gap may go unrecorded.

Progress tracking and current limitations

ss.hist

Topic, level and % score for each quiz.

ss.gaps

Concepts from wrong answers; prompt uses up to 5 recent.

ss.lv

Numeric level, changed by quiz performance.

Session-state only: no database, accounts or sync, so history is lost on restart or reset.

Area	Limitation	Improvement
AI output	May contain incorrect facts	Verification, reference material
Quiz generation	Malformed responses	Stronger validation
Knowledge gaps	Duplicates stored	Deduplicate, rank by frequency
Persistence / accounts	Session-based, no login	SQLite/PostgreSQL, secure login
Personalization	Mostly score-based	Topic-level mastery
Study material	No uploads or grounding	PDF and document ingestion

Install and run AdaptLearn on Windows

1. Install Ollama

Download from the official site and open it.

2. Open in VS Code

Name the file exactly app.py.

3. Virtual env

python -m venv .venv, then activate.

4. Libraries

pip install streamlit requests

5. Get the model

ollama pull llama3.2:3b

6. Verify

ollama list

7. Run

python -m streamlit run app.py, then open localhost:8501

Troubleshooting

"File does not exist": run from the project folder; no app.py.txt.
"Cannot connect to Ollama": make sure it is running and the model is installed.

Future features and summary

PDF-to-learning

Upload notes or lecture PDFs to ground answers.

Learner analytics

Mastery by topic; suggest what to revise.

Flashcards & planner

Revision schedule from exams and missed concepts.

Multi-user platform

Accounts, teacher dashboards, persistent records.

Improvement: move level up only on reliable quiz evidence, e.g. several quizzes, not one 5-question quiz.

AdaptLearn delivers personalized explanations, notes and quizzes using Python, Streamlit and a local Llama 3.2 3B via Ollama. It grades answers, records knowledge gaps and adapts difficulty, making learning more adaptive and accessible. Thank you | Questions?